

Software Design Document
SDD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SDD
Versión	2.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — juanromeroo@javeriana.edu.co
Laura Juliana Garzón Arias — garzonlj@javeriana.edu.co
Xiomara Herrera Suárez — x_herrera@javeriana.edu.co
Sebastián Almanza Galvis — s.almanzag@javeriana.edu.co

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.

Trazabilidad. Este SDD detalla la implementación técnica, estructura modular y diseño funcional de Gamma Lab, en coherencia con el SAD y el SRS previamente definidos.

Índice

1. Introducción	4
1.1. Propósito del documento	4
1.2. Alcance	4
1.3. Audiencia	4
2. Arquitectura	4
2.1. Vista Lógica	4
2.1.1. Propósito	4
2.2. Vista Física	6
2.2.1. Propósito	6
2.2.2. Relaciones	7
2.2.3. Versiones de las librerías	7
2.3. Vista de Procesos	7
2.3.1. Propósito	7
3. Diseño Detallado	9
3.1. Objetivo de la sección	9
3.2. Estructura del Sistema	9
3.3. Componentes del Sistema	9
3.4. Persistencia y Modelos de Datos	9
3.5. Relaciones	9
3.6. Interfaz de Usuario	9
3.6.1. Proposito	9
3.6.2. Organización de la pantalla principal	10
3.6.3. Carga y guardado de archivos (pestaña <i>File</i>)	12
3.6.4. Preprocesamiento	14
3.6.5. Analysis	15

Índice de figuras

1.	Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins. . .	4
2.	Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida.	6
3.	Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema. . .	8
4.	Organización de la pantalla principal de Gamma Lab	10
5.	opciones de slope	10
6.	opciones de eliminar medida	11
7.	opciones de zoom	11
8.	explorador de archivos desplegado	11
9.	resultados desplegados	12
10.	Los resultados pueden ser exportados en formato csv.	12
11.	Pestaña File de Gamma Lab	12
12.	Flujo de carga de la señal (<i>Open Signal</i>)	13
13.	Flujo de carga de proyecto (<i>Open Project</i>)	13
14.	Pestaña Preprocessing de Gamma Lab	14
15.	opciones de trials	14
16.	opciones de remove artifact	15
17.	opciones de filter	15
18.	opciones de FFT	16
19.	opciones de FFT Average	16
20.	opciones de PSD	17
21.	opciones de PSD Average	17
22.	opciones de Average	18
23.	opciones de Event related potential	18
24.	opciones de Wavelet	19
25.	opciones de Wavelet Average	19
26.	opciones de PAC	20

Índice de cuadros

1.	Descripción de componentes de la Vista Lógica	5
----	---	---

Cuadro 1: Descripción de componentes de la Vista Lógica

Componente	Descripción	Relaciones / Dependencias
Kernel (núcleo)	Orquestador central de la aplicación: administra el registro de plugins y servicios, coordina la ejecución y media la comunicación entre módulos.	Provee <code>register_service()/get_service()</code> y <code>register_plugin()</code> ; es consumido por <code>MainWindow</code> y por todos los plugins.
PluginManager / loader	Descubre e instancia los plugins disponibles a partir de sus carpetas y archivos <code>properties.yml</code> .	Utiliza <code>IPlugin</code> como contrato; registra cada instancia en el Kernel.
IPlugin (interfaz)	Contrato que toda extensión del sistema implementa (<code>initialize</code> , <code>start</code> , <code>stop</code> , <code>process</code> , <code>get_widget</code>).	Es implementada por todos los plugins; el Kernel la consume de forma polimórfica.
DataStore	Almacén centralizado en memoria de señales, trials, resultados intermedios y metadatos.	Consumido por el Kernel, los plugins y <code>FileIOService</code> mediante claves canónicas.
FileIOService	Lee archivos de señal en formato ABF, EDF y MAT, y los convierte a la estructura interna <code>SignalDataset</code> .	Utiliza las librerías <code>EEGReaders</code> (<code>pyabf</code> , <code>pyedflib</code>); publica el resultado en <code>DataStore</code> .
ExportService	Genera archivos de salida (imagen y tabla) a partir de las visualizaciones activas.	Utiliza <code>VTK</code> para capturar el render; es consumido por los plugins desde el menú contextual.
Measurement-Service	Gestiona las mediciones interactivas de pendiente y amplitud sobre los gráficos.	Utiliza <code>VTK</code> para la interacción; persiste sus resultados en <code>DataStore</code> .
SettingsService	Administra la configuración general del sistema y las preferencias del usuario.	Persiste en <code>gamma_lab_settings.json</code> ; consumido por el Kernel y los plugins.
ProjectService	Gestiona la persistencia del espacio de trabajo – señal activa, trials y análisis – en archivos de proyecto <code>.glab</code> .	Consume <code>DataStore</code> para serializar y restaurar el estado completo de la sesión.
Task-Orchestrator	Centraliza la ejecución de cómputo pesado (PAC, Wavelet, PSD) fuera del hilo de interfaz, con soporte de cancelación.	Es consumido por los plugins de análisis; se apoya en multiprocesamiento y multithreading.
Plugins (extensible)	Punto de extensión del sistema, organizados por categoría (E/S, Preprocesamiento, Análisis, Medición).	Consumen los servicios del Kernel; dependen de <code>PyQt5</code> y <code>VTK</code> para su interfaz.
MainWindow (APP)	Ventana principal de la aplicación; gestiona la navegación entre categorías de plugins y aloja su interfaz.	Consume el Kernel para descubrir y mostrar los plugins registrados.
PyQt5 (framework)	Framework de interfaz gráfica que administra ventanas, widgets y eventos del usuario.	Utilizado por el Kernel, <code>MainWindow</code> y los plugins para construir la interfaz.
VTK (librería)	Librería de visualización científica 2D/3D.	Usada directamente por los plugins de análisis y por <code>ExportService/MeasurementService</code> .
EEGReaders (librería)	Conjunto de librerías (<code>pyabf</code> , <code>pyedflib</code>) para leer formatos de señal biomédica.	Utilizado por <code>FileIOService</code> .
ScientificLibs (librería)	Conjunto de librerías científicas (<code>NumPy</code> , <code>SciPy</code> , <code>PyWavelets</code>) para el cómputo numérico.	Utilizado por los plugins de análisis y por <code>TaskOrchestrator</code> .

El *Kernel* centraliza el registro y consumo de servicios mediante `register_service()/get_`

`service()`; el *DataStore* soporta la comunicación entre módulos mediante claves canónicas. El código se organiza en paquetes correspondientes a estas mismas responsabilidades (`io`, `pre-processing`, `analysis`, `measure`), lo que mantiene la correspondencia entre el diseño lógico y la implementación.

2.2. Vista Física

2.2.1. Propósito

La vista física representa cómo se despliega y ejecuta GammaLab 2.0. Presenta los componentes físicos más importantes del sistema, sobre los cuales se instalan los componentes de software.

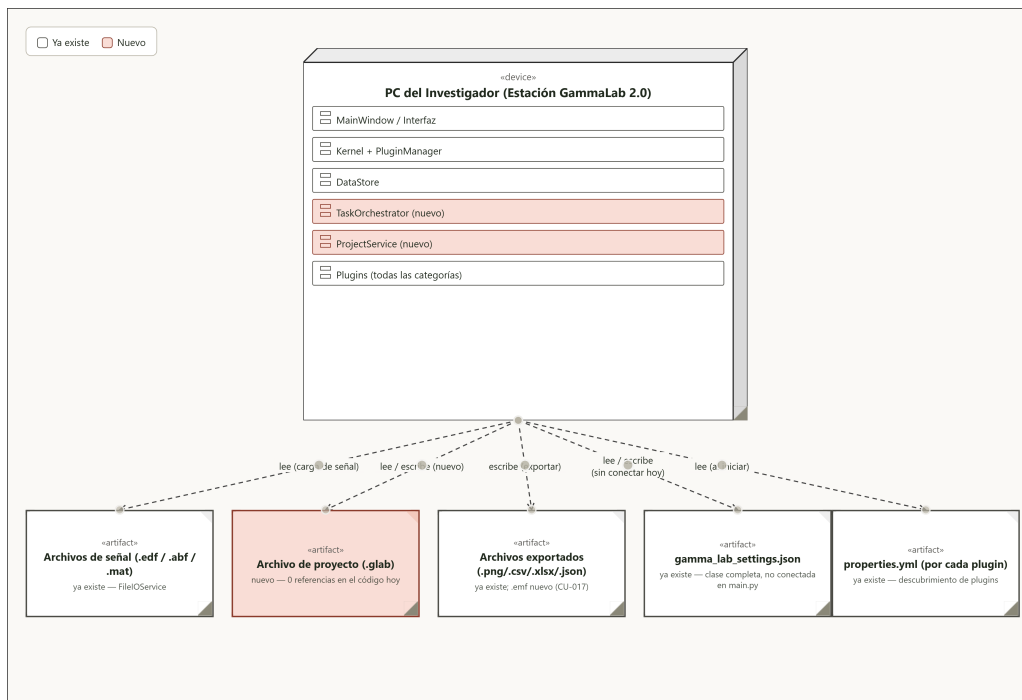


Figura 2: Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida.

La Figura 2 presenta la distribución física de los componentes del sistema en el entorno objetivo de ejecución. A continuación se describen los elementos principales:

- **«device» PC**: estación de trabajo del investigador donde se despliega y ejecuta el sistema completo; no existe arquitectura cliente-servidor.
- **«OS» Windows 10/11**: sistema operativo base; versión mínima Windows 10 de 64 bits.
- **«executionEnvironment» Python Runtime 3.11**: entorno de ejecución que sostiene la aplicación y sus librerías; versión mínima 3.11.x.
- **«artifact» GammaLab.exe**: ejecutable principal, generado con PyInstaller, que inicia la aplicación y coordina la carga de todos los componentes.
- **«artifact» Plugins (*.py)**: archivos de plugin cargados dinámicamente por el Kernel durante la ejecución.
- **«artifact» Archivos de señal (.edf/.abf/.mat)**: leídos por FileIOService.

- «artifact» **Archivo de proyecto** (.glab): generado y leído por `ProjectService`.
- «artifact» **gamma_lab_settings.json**: persistencia de configuración gestionada por `SettingsService`.
- «artifact» **properties.yml** (por cada plugin): sostiene el descubrimiento automático de plugins.

2.2.2. Relaciones

- La relación *loads* indica que el ejecutable principal carga y ejecuta los archivos de plugin durante el tiempo de ejecución.
- La relación *uses* especifica que el ejecutable principal utiliza los servicios y librerías desplegados dentro del mismo entorno de ejecución.

2.2.3. Versiones de las librerías

Las versiones ancladas para las librerías utilizadas, tal como se fijan en `requirements.txt`, son las siguientes:

- **PyQt5** = Versión 5.15.11 (PyQt5-Qt5 5.15.2)
- **ScientificLibs**
 - NumPy = Versión 2.3.4
 - Pandas = Versión 2.3.3
 - SciPy = Versión 1.16.2
 - PyWavelets = Versión 1.9.0
- **VTK** = Versión 9.5.2
- **EEGReaders**
 - pyABF = Versión 2.3.8
 - pyEDFlib = Versión 0.1.42
- **Otras**
 - h5py = Versión 3.15.1
 - PyYAML = Versión 6.0.3
 - PyQtWebEngine = Versión 5.15.7

2.3. Vista de Procesos

2.3.1. Propósito

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondition entre los casos de uso del sistema (p. ej. generar trials exige una señal previamente cargada).

El flujo de la Figura 3 se organiza en las siguientes etapas:

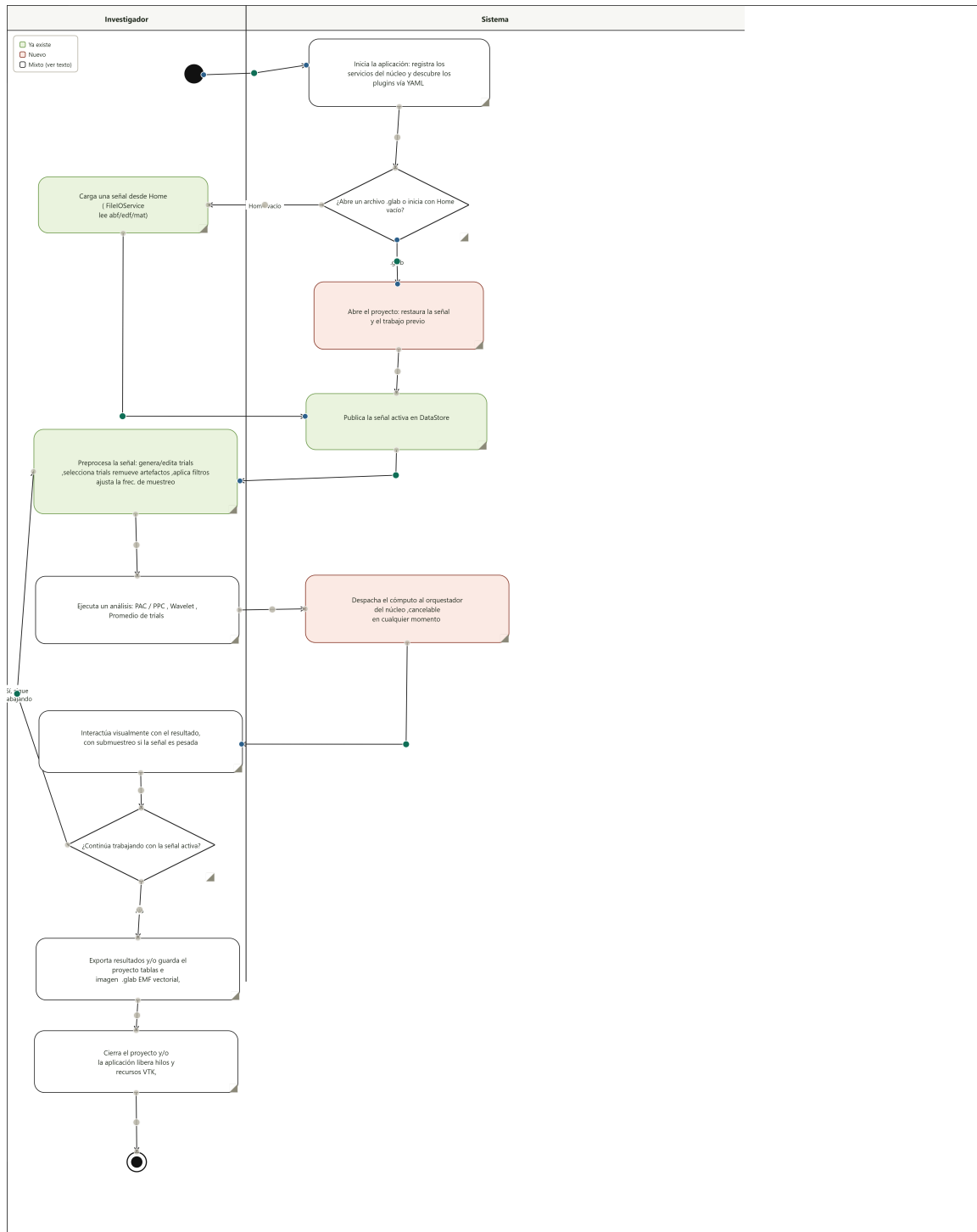


Figura 3: Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema.

1. **Inicio.** El sistema registra los servicios del núcleo, incluido el `TaskOrchestrator`, y descubre los plugins disponibles a partir de sus archivos `properties.yml`.
2. **Apertura o carga.** El investigador abre un archivo de proyecto `.glab` o carga una señal desde la pantalla *Home*; la señal activa se publica en `DataStore`.
3. **Preprocesamiento.** Generación y edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de la frecuencia de muestreo.
4. **Análisis.** Cálculo de acoplamiento fase-amplitud (PAC) de un canal o cruzado entre canales, transformada Wavelet y promedio de trials.
5. **Orquestación.** Todo cómputo pesado se despacha al `TaskOrchestrator` del núcleo, que lo ejecuta fuera del hilo de interfaz y permite su cancelación en cualquier momento.
6. **Visualización.** Interacción visual con el resultado, con submuestreo automático cuando la señal es de gran tamaño.
7. **Iteración o cierre.** El investigador retorna al preprocesamiento para continuar el análisis, o exporta y guarda el proyecto mediante `ProjectService` antes de cerrar la sesión.

La navegación entre secciones, la limpieza de formularios y la ayuda contextual mediante `Tooltip-Helper` son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en vez de forzarse dentro del flujo principal.

3. Diseño Detallado

3.1. Objetivo de la sección

3.2. Estructura del Sistema

3.3. Componentes del Sistema

3.4. Persistencia y Modelos de Datos

3.5. Relaciones

3.6. Interfaz de Usuario

3.6.1. Proposito

Esta sección describe la interfaz de usuario de Gamma Lab, incluyendo la organización, navegación, y componentes de las vistas.

3.6.2. Organización de la pantalla principal

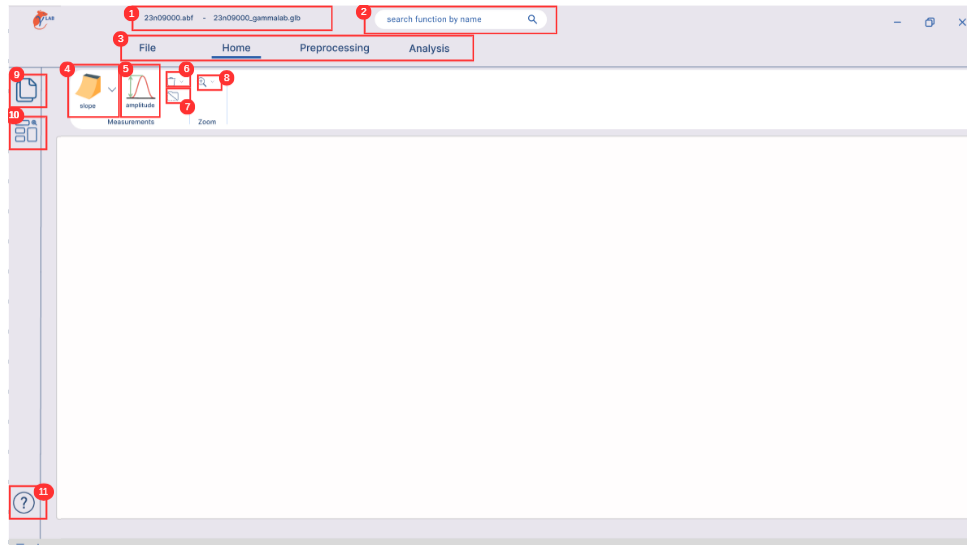


Figura 4: Organización de la pantalla principal de Gamma Lab

La pantalla principal se compone de los siguientes elementos, numerados en la Figura 4:

- **1 - Panel de control:** muestra el nombre de la señal abierta junto al nombre, editable, del proyecto activo, guardado con extensión `.glib`.
- **2 - Barra de búsqueda:** ubicada en la parte superior, permite localizar y acceder rápidamente a una funcionalidad específica del sistema sin tener que recorrer manualmente la barra de navegación.
- **3 - Barra de navegación:** agrupa las pestañas *File*, *Home*, *Preprocessing* y *Analysis*, permitiendo alternar entre la gestión de archivos, las mediciones sobre la señal, el preprocesamiento y los módulos de análisis.
- **4 - slope:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la pendiente de la señal entre dos puntos seleccionados por el usuario, mediante las subopciones *Slope 2 Trials* (calculada sobre dos ensayos) y *Slope All Trials* (calculada sobre la totalidad de los ensayos generados).

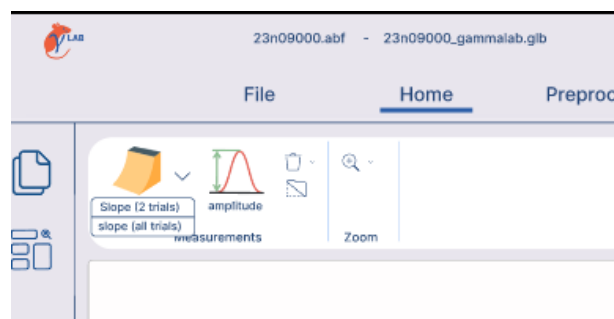


Figura 5: opciones de slope

- **5 - amplitude:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la amplitud de la señal entre dos puntos seleccionados por el usuario sobre la gráfica.

- **6 - eliminar medida:** permite eliminar la última medición de *slope* o *amplitude* realizada, o bien la totalidad de las mediciones registradas sobre la señal.

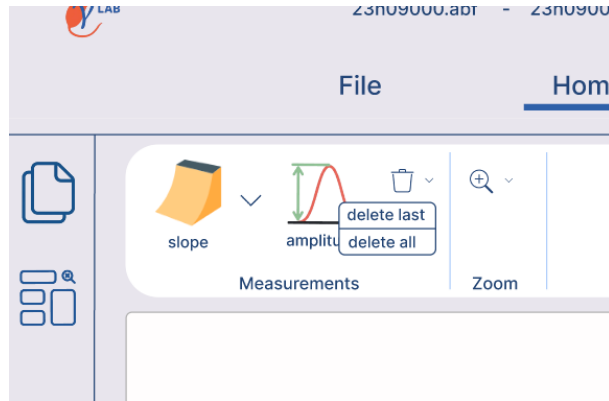


Figura 6: opciones de eliminar medida

- **7 - mostrar/ocultar:** permite mostrar u ocultar en pantalla las mediciones de *slope* y *amplitude* realizadas previamente sobre la señal, sin necesidad de eliminarlas.
- **8 - zoom:** activa o desactiva la función de acercamiento sobre la gráfica de la señal, permitiendo observar con mayor detalle un segmento de la señal y las mediciones realizadas sobre él.



Figura 7: opciones de zoom

- **9 - explorador de archivos:** despliega, desde la barra lateral izquierda, la carpeta del sistema de archivos en la que se encuentra almacenado el proyecto actual junto con las señales y resultados exportados, sin salir de la aplicación.

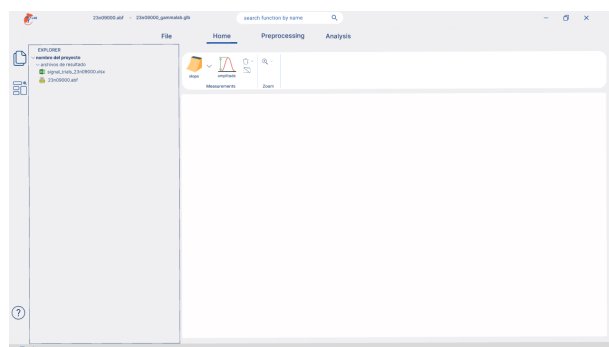


Figura 8: explorador de archivos desplegado

- **10 - resultados:** despliega, desde la barra lateral izquierda, una ventana inferior con los resultados numéricos de las mediciones de *slope* y *amplitude* realizadas sobre la señal, los cuales pueden exportarse en formato *.csv* para su análisis externo.

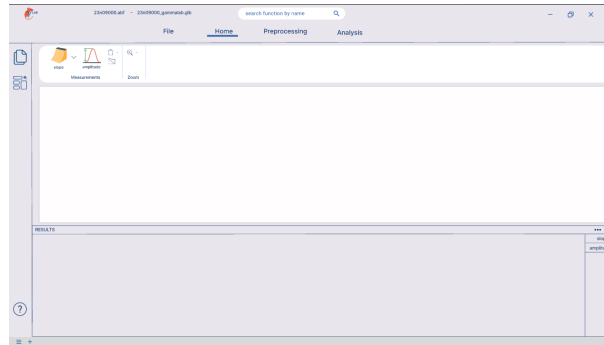


Figura 9: resultados desplegados

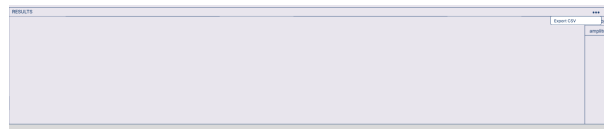


Figura 10: Los resultados pueden ser exportados en formato csv.

- **11 - ayuda:** despliega, desde la barra lateral izquierda, el manual de usuario del sistema, con información de referencia sobre el uso de cada funcionalidad.

3.6.3. Carga y guardado de archivos (pestaña *File*)

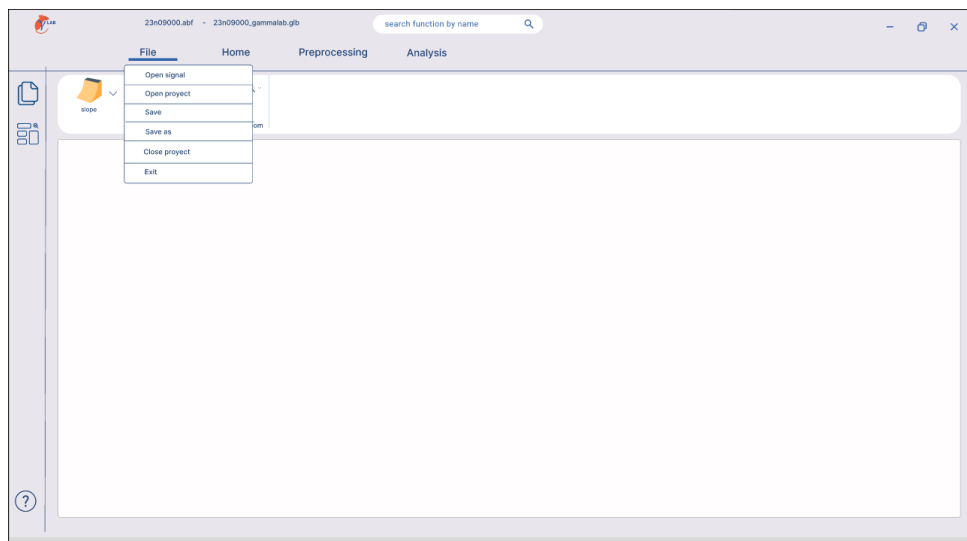


Figura 11: Pestaña File de Gamma Lab

La pestaña *File* permite gestionar la carga y el guardado de señales y proyectos. Agrupa las siguientes opciones:

- **Open Signal:** permite cargar una señal individual dentro de la aplicación para su preprocesamiento y análisis.

- **Open Project:** permite abrir un proyecto previamente guardado con extensión `.glab`, restaurando la señal activa, los trials, los análisis y los parámetros de la sesión.
- **Save:** guarda los cambios realizados sobre el proyecto actualmente abierto.
- **Save As:** permite guardar el proyecto actual bajo un nuevo nombre o ubicación, sin sobrescribir el archivo original.
- **Close Project:** cierra el proyecto que se encuentra abierto, liberando la señal activa de memoria.
- **Exit:** cierra la aplicación.

A continuación se ilustra el flujo de carga de una señal y de un proyecto:

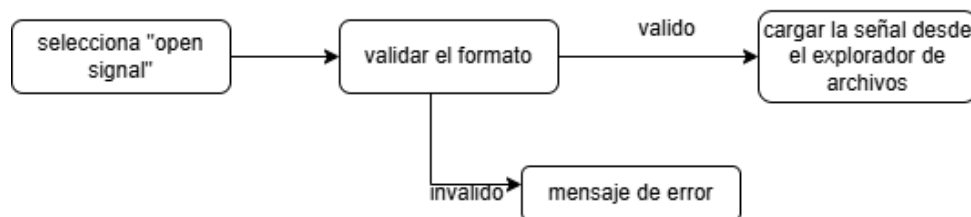


Figura 12: Flujo de carga de la señal (*Open Signal*)

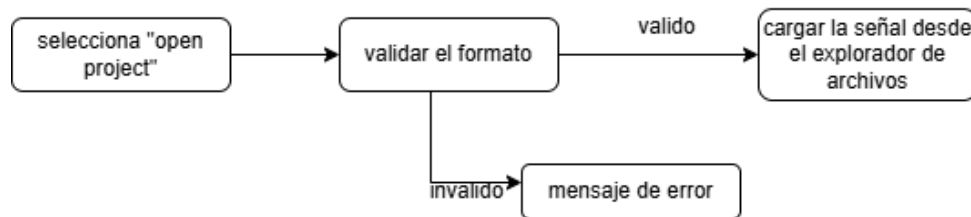


Figura 13: Flujo de carga de proyecto (*Open Project*)

Mediante las opciones *Save* y *Save As* es posible almacenar el estado completo del espacio de trabajo, incluyendo la señal activa, los trials configurados, las medidas realizadas y los resultados de los análisis ejecutados, en un archivo de proyecto con extensión `.glab`.

3.6.4. Preprocesamiento

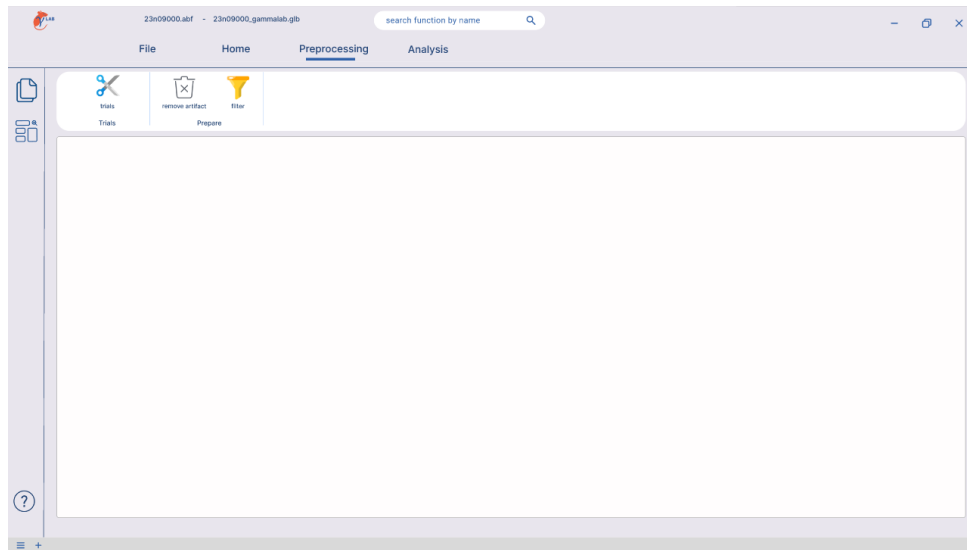


Figura 14: Pestaña Preprocessing de Gamma Lab

Categoría *Trials*

- **trials:** divide la señal cargada en ensayos (*trials*), permitiendo seleccionar el canal a dividir, el canal de estímulos, el umbral (*threshold*), el número de estímulos y el tamaño de la ventana de segmentación. Una vez generados, el usuario puede navegar entre los trials resultantes e incluirlos o descartarlos individualmente del conjunto que participará en los análisis posteriores.

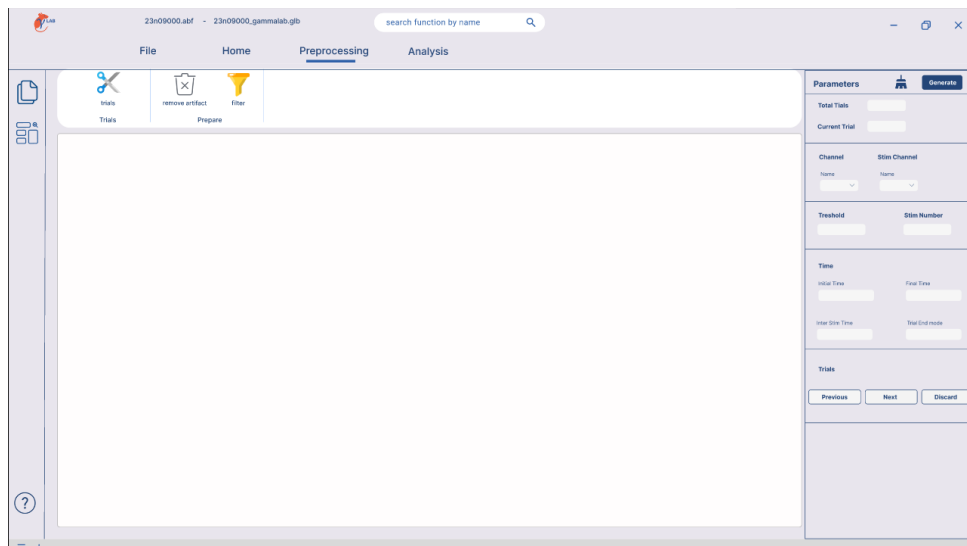


Figura 15: opciones de trials

Categoría *prepare*

- **remove artifact:** elimina artefactos presentes en los trials mediante la selección de un rango de tiempo a corregir, ofreciendo dos modos de corrección: interpolación del segmento seleccionado o eliminación de la señal desde el inicio del trial hasta el punto seleccionado.

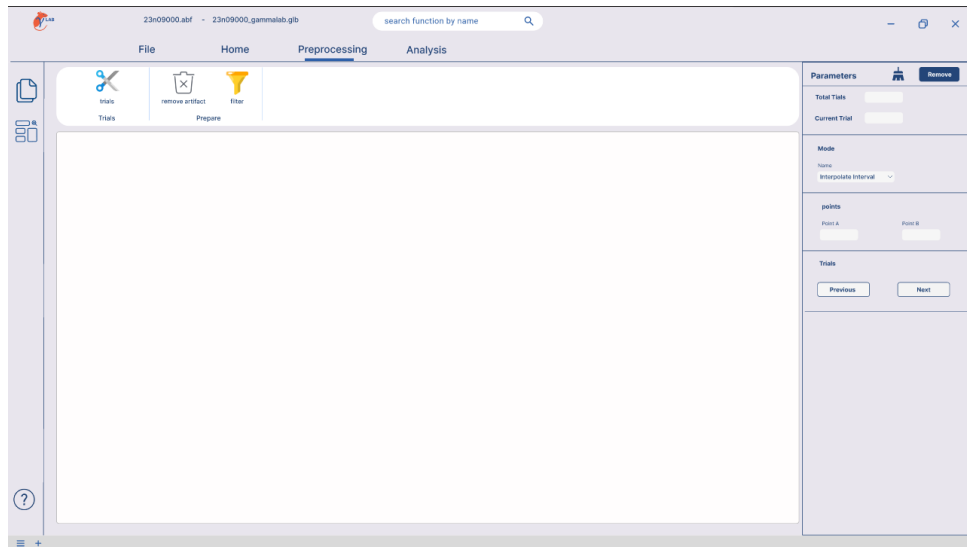


Figura 16: opciones de remove artifact

- **filter:** aplica un filtro digital Butterworth a la señal, mediante la selección del rango de frecuencias de corte y el orden del filtro, mostrando el resultado tanto sobre la señal completa como sobre el primer trial generado.

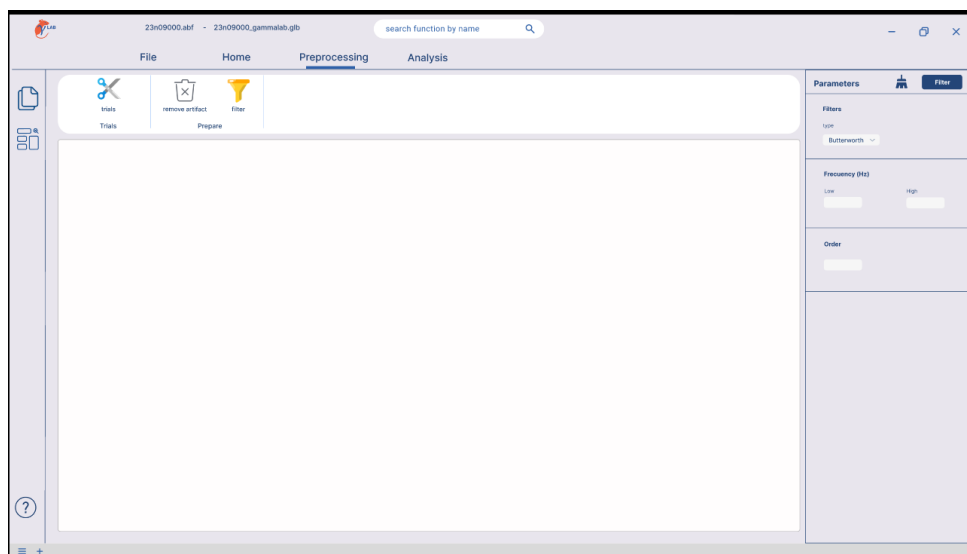


Figura 17: opciones de filter

3.6.5. Analysis

En esta sección se pueden realizar diferentes análisis de la señal, a continuación serán listados los análisis disponibles y sus opciones:

- **FFT:** calcula la transformada rápida de Fourier de la señal, mediante la configuración de la frecuencia de muestreo y el rango de frecuencias de interés, y muestra el espectro de potencia resultante de forma individual para cada ensayo.

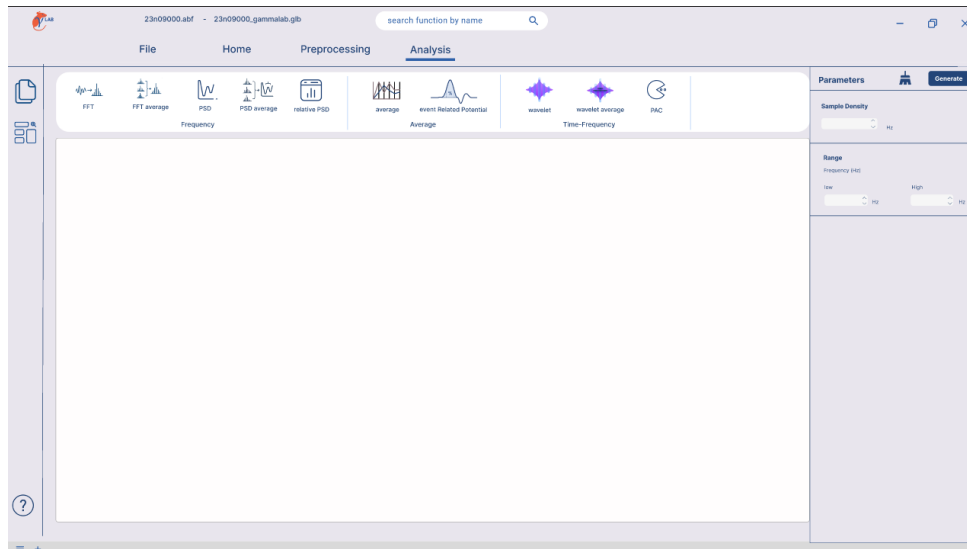


Figura 18: opciones de FFT

- **FFT Average:** calcula la FFT de la señal con la misma configuración de frecuencia de muestreo y rango de frecuencias que la opción anterior, pero adicionalmente promedia los espectros obtenidos de todos los ensayos para reducir el ruido y resaltar los componentes de frecuencia comunes entre ellos.

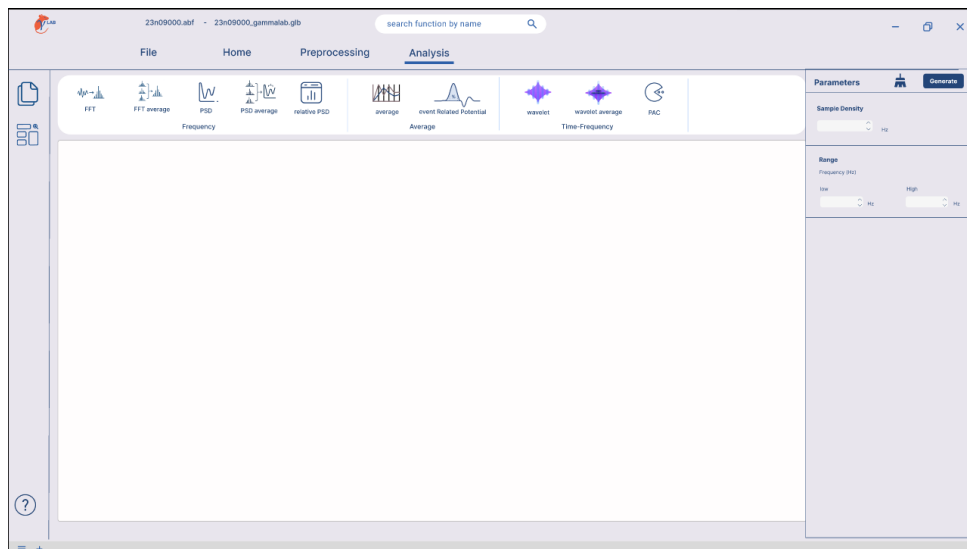


Figura 19: opciones de FFT Average

- **PSD:** estima la densidad espectral de potencia de la señal mediante el método de Welch, permitiendo configurar el modo de cálculo, el tipo de ventana, el rango de frecuencias y el modo de *detrend*, y muestra el resultado de forma individual para cada ensayo.

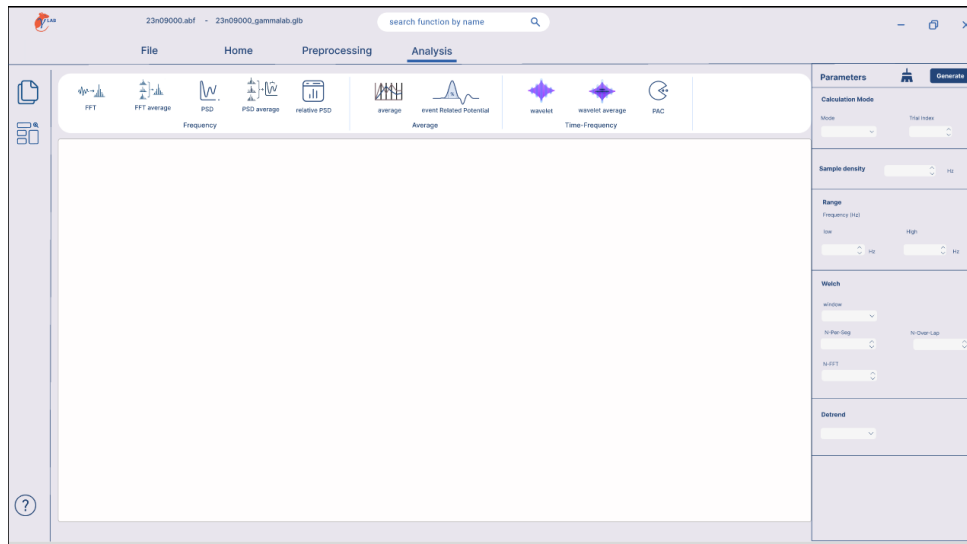


Figura 20: opciones de PSD

- **PSD Average:** estima la PSD de la señal mediante el método de Welch, con la misma configuración de frecuencia de muestreo, ventana y rango de frecuencias que la opción anterior, pero promediando de forma agregada los resultados de todos los ensayos.

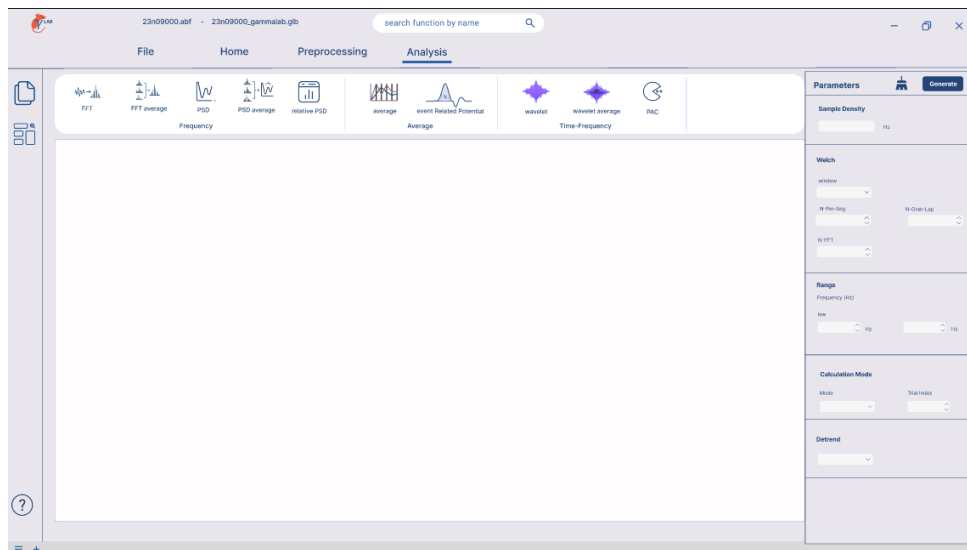


Figura 21: opciones de PSD Average

- **PSD relative:** calcula la PSD relativa de la señal, entregando tanto la potencia absoluta como la potencia relativa aportada por cada banda de frecuencia del rango seleccionado, mediante la configuración del tipo de ventana, el modo de *detrend* y el modo de cálculo.
- **Average:** calcula y visualiza el promedio temporal de los trials generados, útil para observar patrones comunes y atenuar variaciones aleatorias entre ensayos.

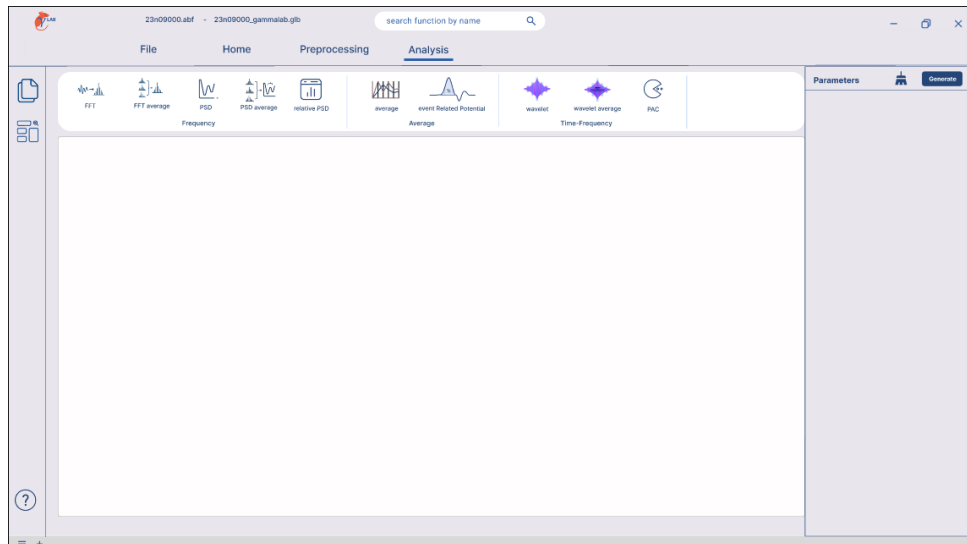


Figura 22: opciones de Average

- **Event related potential:** calcula el potencial relacionado con eventos (ERP), generando un *Butterfly Plot* con todos los trials alineados y un espectrograma de la dinámica tiempo-frecuencia asociada al evento.

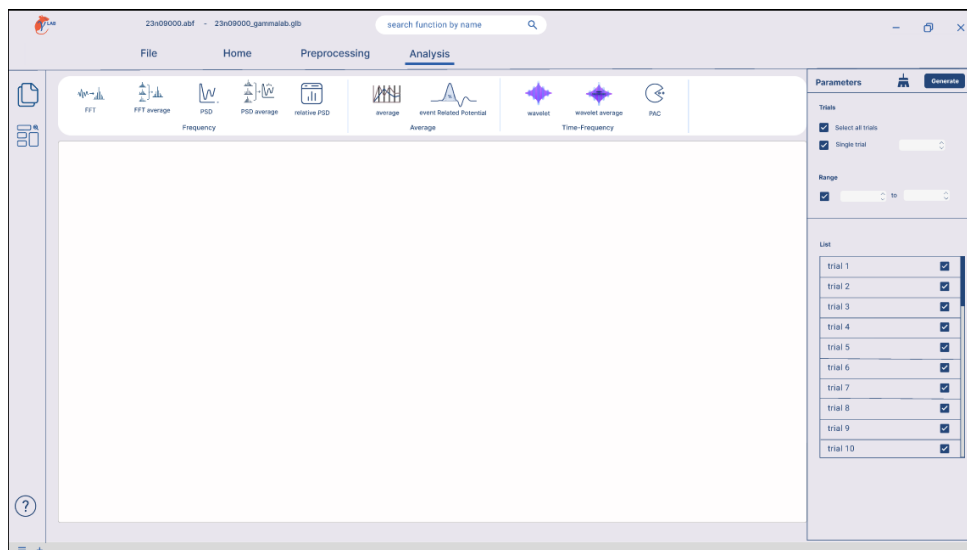


Figura 23: opciones de Event related potential

- **Wavelet:** calcula la transformada wavelet continua de la señal, mediante la configuración de la frecuencia de muestreo, el rango de frecuencias, el número de ciclos, la normalización y la escala de visualización, permitiendo observar cómo varía el contenido espectral de la señal a lo largo del tiempo para cada ensayo.

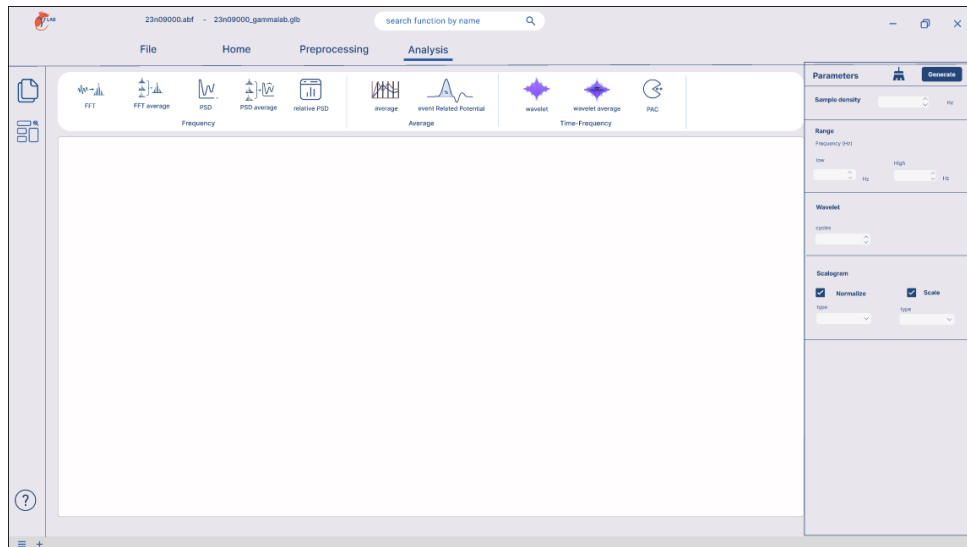


Figura 24: opciones de Wavelet

- **Wavelet Average:** calcula la transformada wavelet con la misma configuración que la opción anterior, y promedia los resultados sobre múltiples trials, generando un mapa tiempo-frecuencia agrupado.

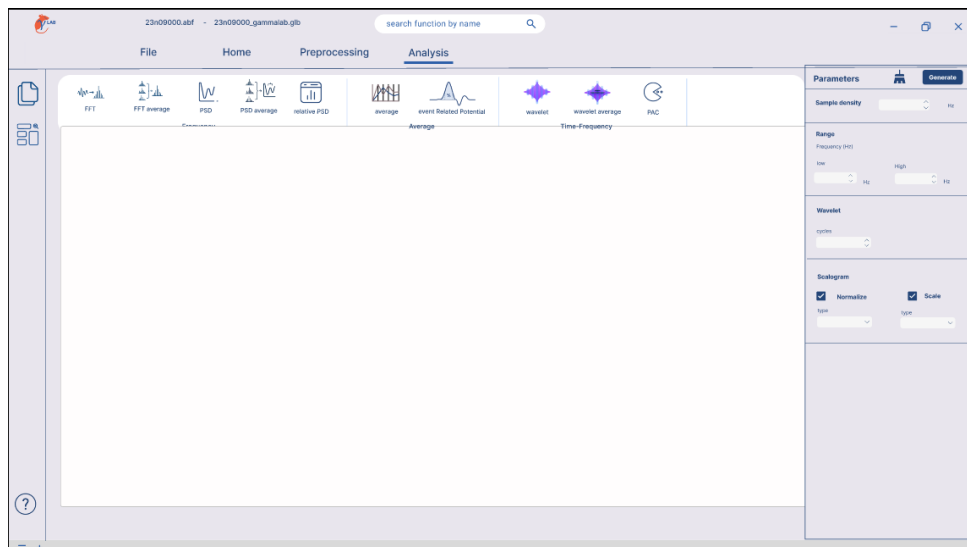


Figura 25: opciones de Wavelet Average

- **PAC:** calcula el acoplamiento de fase-amplitud (*Phase-Amplitude Coupling*) de un canal, evaluando la relación entre la fase de una banda de baja frecuencia y la amplitud de un rango de frecuencias seleccionado, ya sea sobre un trial individual o sobre el promedio de varios trials. El resultado incluye el mapa de acoplamiento fase-frecuencia, el histograma de distribución de amplitud en función de la fase, y el vector de acoplamiento medio con su magnitud y ángulo.

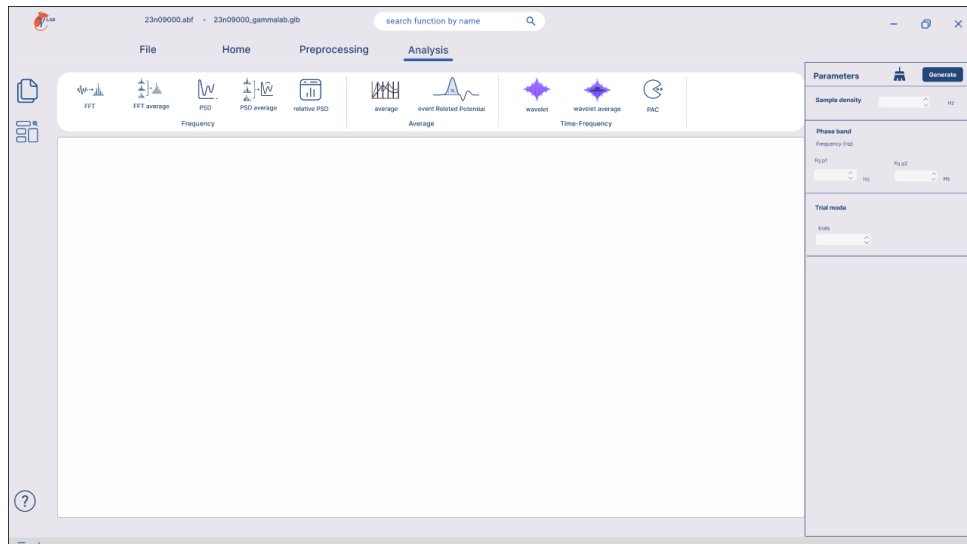


Figura 26: opciones de PAC